Análise de Eficiência Energética e Desempenho da Paralelização do Processamento de Imagens com Filtros Convolucionais

Enzo de Almeida Belfort Rizzi Di Chiara[1], **Luiza de Souza Ferreira**[1], **Natália Vitória Amaral do Val**[1], **Vinícius Augusto Landim Andrade**[1], **Viviane Flor Park**[1]

[1]Instituto de Ciência e Tecnologia – Universidade Federal de São Paulo (UNIFESP)
São José dos Campos – SP – Brasil

{belfort.enzo, souza.luiza, amaral.natalia, vinicius.landim, viviane.park}@unifesp.br

Abstract. *This article describes the implementation of a parallelized version of a convolutional filter algorithm and the corresponding analysis of its energy efficiency and performance. The algorithm is parallelized in two different ways – using the OpenMP library and the CUDA platform – and each version is tested with a set of images of varying dimensions. The results of the parallel implementations are compared against the sequential version of the algorithm and against each other in terms of execution time, speedup, and energy efficiency.*

Resumo. *Este artigo descreve a implementação de uma versão paralelizada de um algoritmo de filtro convolucional e sua respectiva análise de eficiência energética e desempenho. A paralelização do algortimo é feita de duas maneiras diferentes, com a biblioteca OpenMP e a plataforma CUDA, e cada versão é testada com um conjunto de imagens com diferentes dimensões. Os resultados das implementações paralelas são comparados com a versão sequencial do algoritmo e entre si em termos de tempo de execução, speedup e eficiência energética.*

1. Introdução

O crescimento exponencial no volume de dados visuais e a necessidade de processamento em tempo real tornaram o uso de filtros convolucionais uma tarefa de alto custo computacional. Embora a paralelização seja a principal estratégia para acelerar a execução, a eficiência energética tornou-se um fator crítico no projeto de sistemas modernos. O aumento do número de *threads* e o uso intensivo de *hardware* nem sempre geram ganho proporcional de velocidade (*speedup*), podendo elevar drasticamente o consumo de energia devido a gargalos de memória e contenção de barramento.

Este trabalho tem como objetivo, portanto, avaliar o *trade-off* entre ganho de desempenho e eficiência energética na paralelização de filtros convolucionais. Para isso, são analisados diferentes algoritmos e estratégias de concorrência aplicados a imagens de variados tamanhos, mensurando o tempo de execução e o consumo energético do processador. Com isso, busca-se identificar os limites de escalabilidade em que a paralelização otimiza o desempenho mantendo a sustentabilidade energética da aplicação.

2. Fundamentação Teórica e Trabalhos Relacionados

Um filtro convolucional é um mecanismo matemático que opera uma matriz de pesos, chamada de máscara ou *kernel*, sobre os píxeis de uma imagem, alterando os valores originais com base na combinação linear da vizinhança. Devido ao seu elevado número de operações aritméticas repetitivas e à independência entre a computação de píxeis distintos, esta técnica apresenta uma elevada intensidade computacional e um alto grau de paralelismo de dados inerente, tornando-se ideal para arquiteturas concorrentes.

Para explorar este paralelismo, utiliza-se, por exemplo, a biblioteca OpenMP e a plataforma CUDA. O OpenMP é uma biblioteca da linguagem C que possibilita a paralelização de tarefas em sistemas de memória compartilhada. Essa biblioteca encarrega o compilador de distribuir a carga computacional em *threads* que, por sua vez, podem ser processadas em diferentes *cores* da CPU. O CUDA, por outro lado, é uma plataforma da NVIDIA que possibilita a utilização de GPUs para processamento de propósito geral. Dessa maneira, os milhares de núcleos de processamento desse componente passam a executar blocos de *threads* de forma altamente concorrente, reduzindo significativamente a latência de processamento em grandes volumes de dados.

As soluções paralelizadas são frequentemente analisadas e validadas em termos de tempo de execução, *speedup* e, mais recentemente, da sua eficiência energética. O tempo de execução mede a duração total do processamento paralelo e pode ser obtido de forma fácil com funções de auxílio das próprias plataformas utilizadas ou do terminal do sistema operacional da máquina. O *speedup*, por sua vez, é uma métrica que quantifica o ganho em tempo do algoritmo paralelizado (com tempo de execução T_P em P *cores*) em relação à versão sequencial (com tempo de execução T em 1 *core*), e é calculado pela Equação (1).

$$S_P = \frac{T}{T_P} \tag{1}$$

3. Metodologia e Implementação

O algoritmo sequencial usado como base para o trabalho está disponível em `parallel-conv-filter/conv_filter.c`. Estruturalmente, ele pode ser dividido em duas partes principais: o pré-processamento (função `toGray`), que transforma a imagem em tons de cinza, e a convolução (função `convolve`), de fato. Em cima desse algoritmo foram implementadas, então, as versões paralelas com OpenMP e CUDA, detalhadas nas subseções 3.1 e 3.2, respectivamente.

Os testes foram executados em cima de um banco de 100 imagens, classificadas uniformemente em 5 dimensões diferentes (CITAR DIMENSÕES). Essa execução foi feita em um computador com um processador Intel Core i5-9400F com frequência de *clock* de 2.90GHz e 6 *cores*. O computador possui 16GB de memória RAM DDR4 e uma placa de vídeo NVIDIA GeForce GTX 1060 de 6GB.

3.1. Paralelização com OpenMP

A paralelização do algoritmo com OpenMP, disponível em `parallel-conv-filter/openmp/conv_filter.c`, foi realizada em dois momentos distintos: na parte de pré-processamento e de convolução. As duas se

tornaram fortes candidatas à paralelização devido à independência entre o processamento de píxeis diferentes.

Na função de pré-processamento, foi utilizada a diretiva `#pragma omp parallel for`, que permite distribuir as iterações do *loop* entre as *threads*. Desse modo, diferentes píxeis podem ser processados em paralelo. Já na função de convolução, foi utilizada a diretiva `#pragma omp parallel for collapse(2)`. Nesse caso, o elemento `collapse(2)` combina os dois *loops* aninhados, referentes às coordenadas x e y da imagem, em um único espaço de iterações que pode ser distribuído entre as *threads*. Assim, o processamento de píxeis da imagem de saída pode ser realizado em paralelo.

Para chegar na versão final da implementação com OpenMP, foram realizados testes de tempo de execução das funções, comparando diferentes configurações de paralelização. A configuração apresentada foi escolhida com base nos menores tempos de execução obtidos.

3.2. Paralelização com CUDA

4. Resultados e Discussão

5. Conclusão

Referências

[AFIF et al. 2018] AFIF, M., SAID, Y., and ATRI, M. (2018). Efficient 2d convolution filters implementations on graphics processing unit using nvidia cuda. *International Journal of Image, Graphics and Signal Processing*, v. 8.

[CREMEL and ARRUDA 2026] CREMEL, G. and ARRUDA, L. F. (2026). High-performance parallel acceleration of image processing filters on multicore cpus using openmp. *Anais do Computer on the Beach*, v. 17.

[GONZALEZ and WOODS 2010] GONZALEZ, R. and WOODS, R. (2010). *Processamento digital de imagens*. Pearson Education do Brasil, São Paulo, 3 edition.

[HUMAIDAN et al. 2024] HUMAIDAN, B. A., ALGHOFHAILY, S., QHATHANI, M. A., S.OUDAH, and NAGY, N. (2024). Parallel image processing: Taking grayscale conversion using openmp as an example. *Journal of Computer and Communications*, v. 12.